第13章 兵马未动，粮草先行——InnoDB统计数据是如何收集的

我们前面在唠叨查询成本时经常用到一些统计数据，比如通过 SHOW TABLE STATUS 语句可以看到关于表的统计数据，通过SHOW INDEX语句可以看到关于索引的统计数据。那么，这些统计数据是怎么来的呢？它们是以什么方式收集的呢？本章将聚焦于 InnoDB 存储引擎的统计数据收集策略。看完本章后大家就会明白为啥前文老说 InnoDB 的统计信息是不精确的估计值了（言下之意就是我们不打算介绍 MyISAM 存储引擎的统计数据的收集和存储方式，有想了解的读者请自行查阅文档）。

13.1 统计数据的存储方式

InnoDB 提供了两种存储统计数据的方式，分别是永久性地存储统计数据和非永久性地存储统计数据。

- 永久性地存储统计数据：统计数据存储在磁盘上，在服务器重启之后这些统计数据依然存在。
- 非永久性地存储统计数据：统计数据存储在内存中，当服务器关闭时这些统计数据就被清除掉。等到服务器重启之后，在某些适当的场景下会重新收集这些统计数据。

设计 MySQL 的大叔提供了系统变量 innodb_stats_persistent，用来控制将统计数据存储在何处。在 MySQL 5.6.6 版本之前，innodb_stats_persistent 的值默认是 OFF，也就是说 InnoDB 的统计数据默认存储到内存中；自 MySQL 5.6.6 版本起，innodb_stats_persistent 的值默认是 ON，也就是统计数据默认被存储到磁盘上。

不过，InnoDB 默认以表为单位来收集和存储统计数据，也就是说我们可以把某些表的统计数据（以及该表的索引统计数据）存储在磁盘上，把另一些表的统计数据存储在内存中。这是怎么做到的呢？我们可以在创建和修改表的时候，通过指定 STATS_PERSISTENT 属性来指明该表的统计数据的存储方式：

```
CREATE TABLE 表名 (...) Engine=InnoDB, STATS_PERSISTENT = (1|0);

ALTER TABLE 表名 Engine=InnoDB, STATS_PERSISTENT = (1|0);
```

当 STATS_PERSISTENT=1 时，表明想把该表的统计数据永久存储到磁盘上；当 STATS_PERSISTENT=0 时，表明想把该表的统计数据临时存储到内存中。如果在创建表时未指定 STATS_PERSISTENT 属性，则默认采用系统变量 innodb_stats_persistent 的值作为该属性的值。

13.2 基于磁盘的永久性统计数据

当我们选择把某个表以及该表索引的统计数据存放到磁盘上时，实际上是把这些统计数据存储到了两个表中：

```
mysql> SHOW TABLES FROM mysql LIKE 'innodb%stats';
+--------------------------------+
| Tables_in_mysql (innodb%stats) |
+--------------------------------+
| innodb_index_stats             |
| innodb_table_stats             |
+--------------------------------+
2 rows in set (0.02 sec)
```

可以看到，这两个表都位于 mysql 系统数据库下面，其中：

- innodb_table_stats 存储了关于表的统计数据，每一条记录对应着一个表的统计数据；
- innodb_index_stats 存储了关于索引的统计数据，每一条记录对应着一个索引的一个统计项的统计数据。

我们下面的任务就是看看这两个表中都有什么字段，以及表中的数据是如何生成的。

13.2.1 innodb_table_stats

直接看一下 innodb_table_stats 表中的各个列都是干嘛的，如表 13-1 所示。

表 13-1 innodb_table_stats 表中各个列的用途

字段名	描述
database_name	数据库名
table_name	表名
last_update	本条记录最后更新的时间
n_rows	表中记录的条数
clustered_index_size	表的聚簇索引占用的页面数量
sum_of_other_index_sizes	表的其他索引占用的页面数量

注意这个表的主键是（database_name，table_name），也就是 innodb_table_stats 表的每条记录代表着一个表的统计信息。我们直接看一下这个表中的内容：

```
mysql> SELECT * FROM mysql.innodb_table_stats;
+---------------+-----------------+---------------------+--------+----------------------+--------------------------+
| database_name | table_name      | last_update         | n_rows | clustered_index_size | sum_of_other_index_sizes |
+---------------+-----------------+---------------------+--------+----------------------+--------------------------+
| mysql         | gtid_executed   | 2018-07-10 23:51:36 |      0 |                    1 |                        0 |
| sys           | sys_config      | 2018-07-10 23:51:38 |      5 |                    1 |                        0 |
| xiaohaizi     | single_table    | 2018-12-10 17:03:13 |   9693 |                   97 |                      175 |
+---------------+-----------------+---------------------+--------+----------------------+--------------------------+
2 rows in set (0.01 sec)
```

可以看到我们熟悉的 single_table 表的统计信息就对应着 mysql.innodb_table_stats 表的第 3

条记录。几个重要的统计项的值如下。

- n_rows 的值是 9,693，表明 single_table 表中大约有 9,693 条记录。注意这个数据是估计值。
- clustered_index_size 的值是 97，表明 single_table 表的聚簇索引占用 97 个页面。
- sum_of_other_index_sizes 的值是 175，表明 single_table 表的其他索引一共占用 175 个页面。

1. n_rows 统计项的收集

为啥老强调 n_rows 统计项的值是估计值呢？现在就来揭晓答案。InnoDB 在统计一个表中有多少行记录时，套路是这样的：按照一定算法（并不是纯粹随机的）从聚簇索引中选取几个叶子节点页面，统计每个页面中包含的记录数量，然后计算一个页面中平均包含的记录数量，并将其乘以全部叶子节点的数量，结果就是该表的 n_rows 值。

小贴士

真实的计算过程比这个稍微复杂一些，不过大致上就是这样的。

可以看出，这个 n_rows 值精确与否取决于统计时采样的页面数量。设计 MySQL 的大叔很贴心地为我们准备了一个名为 innodb_stats_persistent_sample_pages 的系统变量。在使用永久性的统计数据时，这个系统变量表示计算统计数据时采样的页面数量。该值设置得越大，统计出的 n_rows 值越精确，但是统计耗时也就最久；该值设置得越小，统计出的 n_rows 值越不精确，但是统计耗时就会越少。所以在实际使用时需要我们去权衡利弊。该系统变量的默认值是 20。

前文说过，InnoDB 默认以表为单位来收集和存储统计数据。我们也可以单独设置某个表的采样页面的数量。设置方式就是在创建或修改表时通过指定 STATS_SAMPLE_PAGES 属性，来指明统计该表的信息时使用的采样页面数量：

- CREATE TABLE 表名 (...) Engine=InnoDB, STATS_SAMPLE_PAGES ＝具体的采样页面数量；
- ALTER TABLE 表名 Engine=InnoDB, STATS_SAMPLE_PAGES ＝具体的采样页面数量。

如果在创建表的语句中并没有指定 STATS_SAMPLE_PAGES 属性，将默认使用系统变量 innodb_stats_persistent_sample_pages 的值作为该属性的值。

2. clustered_index_size 和 sum_of_other_index_sizes 统计项的收集

收集这两个统计项的数据时，需要用到之前唠叨的 InnoDB 表空间的大量知识。如果大家压根儿没有看第 9 章，下面的计算过程也就不要看了，看也看不懂。如果看过了那一章的内容，就会发现 InnoDB 表空间的知识真是有用啊！

我们知道一个索引占用 2 个段，每个段由一些零散的页面以及一些完整的区构成。clustered_index_size 代表聚簇索引占用的页面数量，sum_of_other_index_sizes 代表其他索引总共占用的页面数量，所以在收集这两个统计项的数据时，需要统计各个索引对应的叶子节点段和非叶子节点段分别占用的页面数量。而统计一个段占用的页面数量的步骤如下所示。

步骤 1. 从数据字典中找到表的各个索引对应的根页面位置。系统表 SYS_INDEXES 中存储了各个索引对应的根页面信息。

步骤 2. 从根页面的 Page Header 中找到叶子节点段和非叶子节点段对应的 Segment Header。在每个索引的根页面的 Page Header 部分都有两个字段。

- PAGE_BTR_SEG_LEAF：表示 B+ 树叶子节点段的 Segment Header 信息。
- PAGE_BTR_SEG_TOP：表示 B+ 树非叶子节点段的 Segment Header 信息。

Segment Header 的结构示意图如图 13-1 所示。

步骤 3. 从叶子节点段和非叶子节点段的 Segment Header 中找到这两个段对应的 INODE Entry 结构。

INODE Entry 的结构示意图如图 13-2 所示。

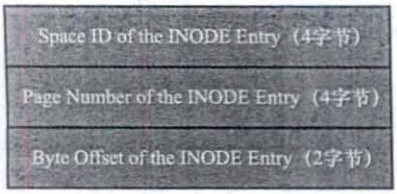

图 13-1 Segment Header 结构示意图

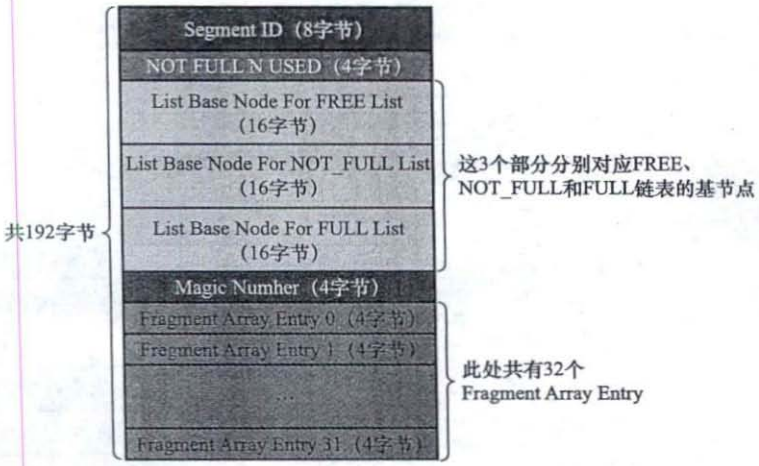

图 13-2 INODE Entry 结构示意图

步骤 4. 针对某个段对应的 INODE Entry 结构，从中找出该段对应的所有零散页面的地址以及 FREE、NOT_FULL 和 FULL 链表的基节点。

链表基节点的示意图如图 13-3 所示。

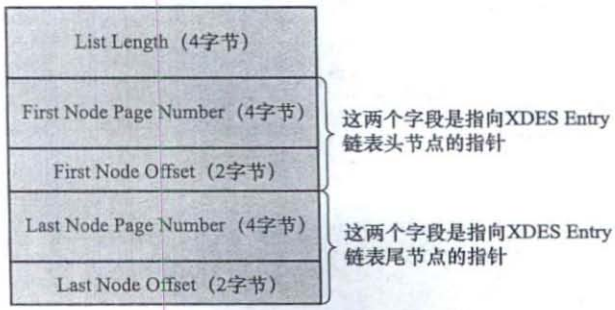

图 13-3 链表基节点的示意图

步骤 5. 直接统计零散的页面有多少个，然后从 FREE、NOT_FULL、FULL 这 3 个链表的 List Length 字段中读出该段占用的区的数量。每个区占用 64 个页，所以就可以统计出整个段占用的页面。

通过上面 5 个步骤，我们可以轻松地将索引的某个段占用的页面数量统计出来，然后分别计

算聚簇索引的叶子节点段和非叶子节点段占用的页面数，它们的和就是 clustered_index_size 的值。按照同样的套路把其余索引占用的页面数都算出来，相加之后就是 sum_of_other_index_sizes 的值。

这里需要注意的是，当一个段的数据非常多时（超过 32 个页面），会以区为单位来申请空间。这里的问题是以区为单位申请空间后，有一些页可能并没有使用，但是在统计 clustered_index_size 和 sum_of_other_index_sizes 时都把它们算进去了，所以聚簇索引和其他索引实际占用的页面数可能比这两个统计项的值要小一些。

13.2.2　innodb_index_stats

直接看一下 innodb_index_stats 表中的各个列都是干嘛的，如表 13-2 所示。

表 13-2　innodb_index_stats 表中各个列的用途

字段名	描述
database_name	数据库名
table_name	表名
index_name	索引名
last_update	本条记录最后更新的时间
stat_name	统计项的名称
stat_value	对应的统计项的值
sample_size	为生成统计数据而采样的页面数量
stat_description	对应的统计项的描述

注意这个表的主键是（database_name，table_name，index_name，stat_name），其中 stat_name 是指统计项的名称，也就是说 innodb_index_stats 表中的每条记录代表着一个索引的一个统计项。可能大家现在不知道这个统计项到底指什么。别着急，我们直接看一下关于 single_table 表的索引统计数据都有些什么。

```
mysql> SELECT * FROM mysql.innodb_index_stats WHERE table_name = 'single_table';
+---------------+-------------+-------------+---------------------+-------------+------------+-------------+-----------------------------------+
| database_name | table_name  | index_name  | last_update         | stat_name   | stat_value | sample_size | stat_description                  |
+---------------+-------------+-------------+---------------------+-------------+------------+-------------+-----------------------------------+
| xiaohaizi     | single_table | PRIMARY     | 2018-12-14 14:24:46 | n_diff_pfx01 |       9693 |          20 | id                                |
| xiaohaizi     | single_table | PRIMARY     | 2018-12-14 14:24:46 | n_leaf_pages |         91 |        NULL | Number of leaf pages in the index |
| xiaohaizi     | single_table | PRIMARY     | 2018-12-14 14:24:46 | size         |         97 |        NULL | Number of pages in the index      |
| xiaohaizi     | single_table | idx_key1    | 2018-12-14 14:24:46 | n_diff_pfx01 |        968 |          28 | key1                              |
| xiaohaizi     | single_table | idx_key1    | 2018-12-14 14:24:46 | n_diff_pfx02 |      10000 |          28 | key1,id                           |
| xiaohaizi     | single_table | idx_key1    | 2018-12-14 14:24:46 | n_leaf_pages |         28 |        NULL | Number of leaf pages in the index |
| xiaohaizi     | single_table | idx_key1    | 2018-12-14 14:24:46 | size         |         29 |        NULL | Number of pages in the index      |
| xiaohaizi     | single_table | idx_key3    | 2018-12-14 14:24:46 | n_diff_pfx01 |        799 |          31 | key3                              |
| xiaohaizi     | single_table | idx_key3    | 2018-12-14 14:24:46 | n_diff_pfx02 |      10000 |          31 | key3,id                           |
| xiaohaizi     | single_table | idx_key3    | 2018-12-14 14:24:46 | n_leaf_pages |         31 |        NULL | Number of leaf pages in the index |
| xiaohaizi     | single_table | idx_key3    | 2018-12-14 14:24:46 | size         |         32 |        NULL | Number of pages in the index      |
| xiaohaizi     | single_table | idx_key_part | 2018-12-14 14:24:46 | n_diff_pfx01 |       9673 |          64 | key_part1                         |
| xiaohaizi     | single_table | idx_key_part | 2018-12-14 14:24:46 | n_diff_pfx02 |       9999 |          64 | key_part1,key_part2               |
| xiaohaizi     | single_table | idx_key_part | 2018-12-14 14:24:46 | n_diff_pfx03 |      10000 |          64 | key_part1,key_part2,key_part3     |
| xiaohaizi     | single_table | idx_key_part | 2018-12-14 14:24:46 | n_diff_pfx04 |      10000 |          64 | key_part1,key_part2,key_part3,id  |
| xiaohaizi     | single_table | idx_key_part | 2018-12-14 14:24:46 | n_leaf_pages |         64 |        NULL | Number of leaf pages in the index |
| xiaohaizi     | single_table | idx_key_part | 2018-12-14 14:24:46 | size         |         97 |        NULL | Number of pages in the index      |
| xiaohaizi     | single_table | uk_key2     | 2018-12-14 14:24:46 | n_diff_pfx01 |      10000 |          16 | key2                              |
| xiaohaizi     | single_table | uk_key2     | 2018-12-14 14:24:46 | n_leaf_pages |         16 |        NULL | Number of leaf pages in the index |
| xiaohaizi     | single_table | uk_key2     | 2018-12-14 14:24:46 | size         |         17 |        NULL | Number of pages in the index      |
+---------------+-------------+-------------+---------------------+-------------+------------+-------------+-----------------------------------+
20 rows in set (0.03 sec)
```

这个结果有点多，正确查看这个结果的方式是这样的。

步骤 1. 先查看 index_name 列，这个列用来说明该记录是哪个索引的统计信息。从结果中可以看出，PRIMARY 索引（也就是主键）占了 3 条记录，idx_key_part 索引占了 6 条记录。

步骤 2. 针对 index_name 列相同的记录，stat_name 表示针对该索引的统计项名称，stat_value 表示的是该索引在该统计项上的值，stat_description 用来描述该统计项的含义。

下面具体看看一个索引都有哪些统计项。

- n_leaf_pages：表示该索引的叶子节点实际占用多少页面。
- size：表示该索引共占用多少页面（包括已经分配给叶子节点段或者非叶子节点段但是尚未使用的页面）。
- n_diff_pfxNN：表示对应的索引列不重复的值有多少。其中的 NN 长得有点儿怪呀，它表示啥意思呢？其实 NN 可以被替换为 01、02、03…… 这样的数字。比如对于 idx_key_part 来说：
  - n_diff_pfx01 表示的是统计 key_part1 这一个列中不重复的值有多少；
  - n_diff_pfx02 表示的是统计 key_part1、key_part2 这两个列组合起来后不重复的值有多少；
  - n_diff_pfx03 表示的是统计 key_part1、key_part2、key_part3 这 3 个列组合起来后不重复的值有多少；
  - n_diff_pfx04 表示的是统计 key_part1、key_part2、key_part3、id 这 4 个列组合起来后不重复的值有多少。

小贴士

这里需要注意的是，对于普通的二级索引，并不能保证它的索引列值是唯一的。比如对于 idx_key1 来说，key1 列就可能有很多值重复的记录。此时只有在索引列的基础上加上主键，才可以区分两条索引列值都一样的二级索引记录。对于主键和唯一二级索引则没有这个问题，它们本身就可以保证索引列的值是不重复的，所以也不需要再统计一遍在索引列后加上主键值后的不重复值有多少。比如前文的 idx_key1 有 n_diff_pfx01 和 n_diff_pfx02 两个统计项，其中的 n_diff_pfx02 表示的就是统计 key1 和 id 这两个列组合起来不重复的值有多少；而 uk_key2 却只有 n_diff_pfx01 一个统计项，该统计项表示的就是 key2 列不重复的值有多少。

步骤 3. 在计算某些索引列中包含多少个不重复的值时，需要对一些叶子节点页面进行采样。sample_size 列就表明了采样的页面数量是多少。

小贴士

对于有多个列的联合索引来说，需要采样的页面数量是 innodb_stats_persistent_sample_pages × 索引中包含的列的个数。当需要采样的页面数量大于该索引的叶子节点的数量时，那么所有的叶子节点都需要被采样。大家可以在查询结果中看到，不同索引对应的 sample_size 列的值可能是不同的。

13.2.3 定期更新统计数据

随着我们不断对表进行增删改操作，表中的数据也一直在变化。innodb_table_stats 和 innodb_index_stats 表中的统计数据是不是也应该跟着变化呢？当然要变了，如果不变，MySQL 优化

器计算出的成本可就很不准确了。设计 MySQL 的大叔提供了下面两种更新统计数据的方式。

- 开启 innodb_stats_auto_recalc

系统变量 innodb_stats_auto_recalc 决定了服务器是否自动重新计算统计数据。它的默认值是 ON，也就是该功能默认是开启的。每个表都维护了一个变量，该变量记录着对该表进行增删改的记录条数。如果发生变动的记录数量超过了表大小的 10%，并且自动重新计算统计数据的功能是打开的，那么服务器会重新计算一次统计数据，并且更新 innodb_table_stats 和 innodb_index_stats 表。不过自动重新计算统计数据的过程是异步发生的，也就是即使表中变动的记录数超过了 10%，可能也不会立即自动重新计算统计数据，因为存在一定的延迟。

再一次强调，InnoDB 默认以表为单位来收集和存储统计数据。我们也可以单独为某个表设置是否自动重新计算统计数据的属性。设置方式就是在创建或修改表时，通过指定 STATS_AUTO_RECALC 属性来指明是否为这个表自动重新计算统计数据：

- CREATE TABLE 表名 (...) Engine = InnoDB, STATS_AUTO_RECALC = (1|0);
- ALTER TABLE 表名 Engine = InnoDB, STATS_AUTO_RECALC = (1|0);

当 STATS_AUTO_RECALC = 1 时，表明想让该表自动重新计算统计数据；当 STATS_AUTO_RECALC = 0 时，表明不想让该表自动重新计算统计数据。如果在创建表时未指定 STATS_AUTO_RECALC 属性，则默认采用系统变量 innodb_stats_auto_recalc 的值作为该属性的值。

- 手动调用 ANALYZE TABLE 语句来更新统计信息

我们也可以手动调用 ANALYZE TABLE 语句重新计算统计数据。比如下面的语句就是用来更新 single_table 表的统计数据：

```
mysql> ANALYZE TABLE single_table;
+------------------------+---------+----------+----------+
| Table                  | Op      | Msg_type | Msg_text |
+------------------------+---------+----------+----------+
| xiaohaizi.single_table | analyze | status   | OK       |
+------------------------+---------+----------+----------+
1 row in set (0.08 sec)
```

需要注意的是，ANALYZE TABLE 语句会立即重新计算统计数据，也就是这个过程是同步的。在表中索引较多或者采样页面特别多时，这个过程可能会比较慢。

13.2.4　手动更新 innodb_table_stats 和 innodb_index_stats 表

其实 innodb_table_stats 和 innodb_index_stats 表与普通的表别无二致，我们也能对它们执行增删改查操作。这也就意味着我们可以手动更新某个表或者索引的统计数据。比如，我们想更改 single_table 表中关于行数的统计数据，可以按照下面的步骤进行操作。

步骤 1. 更新 innodb_table_stats 表。

```
UPDATE innodb_table_stats
    SET n_rows = 1
    WHERE table_name = 'single_table';
```

步骤 2. 让 MySQL 优化器重新加载更改后的数据。

更新后的 innodb_table_stats 只是单纯地修改了一个表的数据。MySQL 优化器需要重新加

载更改过的数据，为此可以运行下面的命令：

```
FLUSH TABLE single_table;
```

之后在使用 SHOW TABLE STATUS 语句查看表的统计数据时，就看到 Rows 行变为了 1。

13.3 基于内存的非永久性统计数据

当把系统变量 innodb_stats_persistent 的值设置为 OFF 时，之后创建的表的统计数据默认就都是非永久性的了。或者我们在创建或修改表时，直接将 STATS_PERSISTENT 属性的值设置为 0，那么该表的统计数据就是非永久性的了。

与永久性的统计数据不同，非永久性的统计数据采样的页面数量是由系统变量 innodb_stats_transient_sample_pages 来控制的，它的默认值是 8。

非永久性的统计数据在每次服务器关闭时以及执行某些操作后会被清除，并在下次访问表时重新计算。这样就可能导致在重新计算统计数据时得到不同的结果，从而可能生成经常变化的执行计划，让用户发懵。不过，最近的 MySQL 版本都不怎么使用这种基于内存的非永久性统计数据了，所以我们也就不深入唠叨它了。

13.4 innodb_stats_method 的使用

我们知道，索引列中不重复的值的数量对于 MySQL 优化器十分重要，通过它可以计算出在索引列中一个值平均重复多少行。它的应用场景主要有两个。

- 单表查询中的单点扫描区间太多。

比如在下面的命令中：

```
SELECT * FROM tbl_name WHERE key IN ('xx1', 'xx2', ..., 'xxn');
```

当 IN 语句对应的单点扫描区间太多时，采用 index dive 的方式直接访问 B+ 树索引，来统计每个单点扫描区间对应的记录的数量就太耗费性能了。所以直接依赖统计数据中一个值平均重复多少行来计算单点扫描区间对应的记录数量。

- 在执行连接查询时，如果有涉及两个表的等值匹配连接条件，该连接条件对应的被驱动表中的列又拥有索引时，则可以使用 ref 访问方法来查询被驱动表。

比如在下面的语句中：

```
SELECT * FROM t1 JOIN t2 ON t1.column = t2.key WHERE ...;
```

假设 t2 为被驱动表，在优化器生成执行计划时，查询并没有真正执行。也就是说，在真正执行对 t2 表的查询前，t1.column 的值是不确定的。我们不能使用 index dive 的方式直接访问 B+ 树索引，来统计 t2 表索引的每个单点扫描区间对应的记录数量。也就是说，我们只能依赖统计数据中一个值平均重复多少行来计算单点扫描区间对应的记录数量。

在统计索引列中不重复的值的数量时，有一个比较棘手的问题是：当索引列中出现 NULL 值时该怎么办？比如，某个索引列的内容是下面这样的：

```
+------+
| col  |
+------+
|    1 |
|    2 |
| NULL |
| NULL |
+------+
```

此时在计算这个 col 列中不重复的值的数量时，就存在下面的分歧。

- 有人认为 NULL 值代表一个未确定的值，所以设计 MySQL 的大叔才认为任何与 NULL 值进行比较的表达式的值都为 NULL。比如下面这样：

```
……
mysql> SELECT 1 = NULL;
+----------+
| 1 = NULL |
+----------+
|     NULL |
+----------+
1 row in set (0.00 sec)

mysql> SELECT 1 != NULL;
+-----------+
| 1 != NULL |
+-----------+
|      NULL |
+-----------+
1 row in set (0.00 sec)

mysql> SELECT NULL = NULL;
+-------------+
| NULL = NULL |
+-------------+
|        NULL |
+-------------+
1 row in set (0.00 sec)

mysql> SELECT NULL != NULL;
+--------------+
| NULL != NULL |
+--------------+
|         NULL |
+--------------+
1 row in set (0.00 sec)
……
```

所以每一个 NULL 值都是独一无二的，也就是说在统计索引列中不重复的值的数量时，

应该把 NULL 值当作一个独立的值，所以 col 列中不重复的值的数量就是 4（分别是 1、2、NULL、NULL 这 4 个值）。

- 有人认为其实 NULL 值在业务上就是表示“没有”，因此所有的 NULL 值代表的意义是一样的。这样一来，col 列中不重复的值的数量就是 3（分别是 1、2、NULL 这 3 个值）。
- 有人认为 NULL 完全没有意义，所以在统计索引列中不重复的值的数量时压根儿就不能把它们算进来，所以 col 列中不重复的值的数量就是 2（分别是 1、2 这两个值）。

设计 MySQL 的大叔蛮贴心的，他们提供了一个名为 innodb_stats_method 的系统变量。这个值的作用是，在计算某个索引列中不重复值的数量时，将“如何对待 NULL 值”的这口锅甩给用户。这个系统变量有 3 个候选值。

- nulls_equal：认为所有 NULL 值都是相等的。这个值也是 innodb_stats_method 的默认值。

如果某个索引列中的 NULL 值特别多，这种统计方式会让查询优化器认为某个列中一个值的平均重复次数特别多，所以倾向于不使用索引进行访问。

- nulls_unequal：认为所有 NULL 值都是不相等的。

如果某个索引列中的 NULL 值特别多，这种统计方式会让查询优化器认为某个列中一个值的平均重复次数特别少，所以倾向于使用索引进行访问。

- nulls_ignored：直接把 NULL 值忽略掉。

反正这口锅是甩给用户了。当选定了 innodb_stats_method 值之后，查询优化器即使选择了不是最优的执行计划，那也与设计 MySQL 的大叔没关系了。

小贴士

上述关于 innodb_stats_method 系统变量的知识是在 MySQL 文档中体现的，但是当我实际查看 MySQL 5.7.22 的代码时出现了一个让我十分迷惑的地方。就是对于基于内存的统计数据来说，nulls_unequal 和 nulls_ignored 的效果是一样的；而对于基于磁盘的统计数据来说，无论我们将 innodb_stats_method 设置为什么值，都将其认为是 nulls_equal（直接硬编码到代码中）。设计 InnoDB 的大叔为啥这么做呢？我也不清楚。

13.5 总结

InnoDB 以表为单位来收集统计数据。这些统计数据可以是基于磁盘的永久性统计数据，也可以是基于内存的非永久性统计数据。

innodb_stats_persistent 控制着服务器使用永久性统计数据还是非永久性统计数据，innodb_stats_persistent_sample_pages 控制着永久性统计数据的采样页面数量，innodb_stats_transient_sample_pages 控制着非永久性统计数据的采样页面数量，innodb_stats_auto_recalc 控制着是否自动重新计算统计数据。

我们可以在创建和修改表时通过指定 STATS_PERSISTENT、STATS_AUTO_RECALC、STATS_SAMPLE_PAGES 的值来控制收集统计数据时的一些细节。

innodb_stats_method 决定着在统计某个索引列中不重复的值的数量时如何对待 NULL 值。